



Fluent, Fast, and Fake

.NET Synthetic FHIR Data for Developer Happiness

Brendan Kowitz, Microsoft

HL7 FHIR DevDays

The largest FHIR-only
event in the world

Minneapolis, MN

www.devdays.com



HL7 FHIR DevDays 2026 | Minneapolis, MN | June 15-18, 2026 | @HL7 | @FirelyTeam | #fhirdevdays | www.devdays.com

ORGANIZED BY



Who am I?

- **Brendan Kowitz**
- EM for FHIR OSS, Azure Health Data Services (FHIR)
- Originally from Australia; lived in the Pacific Northwest for the past 10+ years
- github.com/brendankowitz



GitHub

github.com/brendankowitz/FHIRDevDays2026

THE PROBLEM

Every FHIR dev hits this wall. You need test data.

You're writing a test. You need a Patient with type-2 diabetes — 3 conditions, 2 medications, realistic vitals over 5 years. **Every option in front of you feels a little unsatisfying.**

1

Hand-craft JSON

Hours of work. Breaks on every model change. Zero clinical realism.

2

Bespoke scripts / generator??

Likely produces unrealistic data or hard to model

3

Fire up Synthea

JVM dependency. Minutes per run.
Built for cohorts, not unit tests.

There is a better way — built for .NET, by a .NET developer.

THE REQUIREMENTS NOBODY WROTE DOWN

What developers actually need.

Requirement	Hand-crafted	Synthea	FhirFakes
Fast	✓	×	✓
Can be deterministic per test	✓	×	✓
Native .NET / NuGet	✓	×	✓
Realistic options	×	✓	✓
Fluent C# API	×	×	✓
FHIR-valid output	×	✓	✓
Population-scale	×	✓	✓

The missing piece between static JSON and Synthea

INTRODUCING

Ignixa.FhirFakes

Native .NET synthetic FHIR data — **fluent, fast, and deterministic.**

Native .NET 9/10

No JVM. Generates in milliseconds.

FHIR-valid

Passes schema + binding validation.

Multi-version

STU3 · R4 · R4B · R5 · R6-ballot2

MIT · open source

Inspired by Synthea.

```
$ dotnet add package Ignixa.FhirFakes  
  
$ dotnet tool install -g Ignixa.FhirFakes.Cli
```

What is “Ignixa”

Ignixa started as a personal open-source project for rethinking FHIR/.NET from a clean slate.



Research

A proving ground for new FHIR/.NET ideas, built to learn and explore. Tackling even advanced parts of the spec.



Architecture

Rethinks FHIR on modern .NET: Ground up Parsing, Specification, Serialization, FHIRPath with clean, modular packages.

Publishing 20+ nuget packages. FHIRPath Engine has 45K downloads

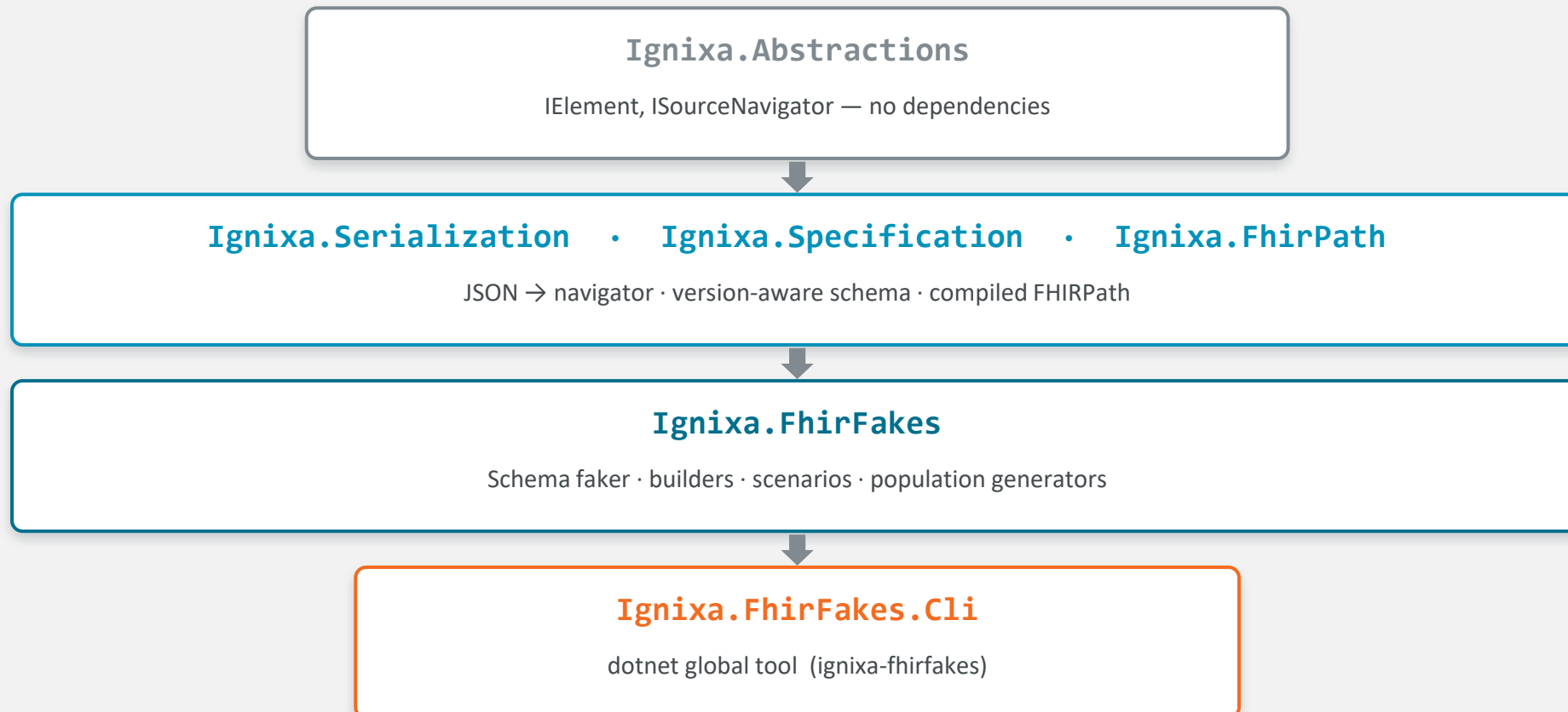


Built with frontier AI

8+ years of my own experience, distilled into code with the latest AI coding tools.

Modular. Pick what you need.

PACKAGE ANATOMY



THE CORE DESIGN

Four layers. Pick your level.

1	Layer 1 • Random Valid Resources Schema-driven, binding-aware	A valid Patient resource in three lines
2	Layer 2 • Clinical Scenarios Fluent C# — express clinical intent	Wellness visit, 15% chance of hyperlipidemia
3	Layer 3 • Patient Lifecycles A full life simulated, birth to today	50-year-old with metabolic syndrome, full history
4	Layer 4 • Population Generation Realistic cohorts — US Census demographics	<code>ignixa-fakes r4 population --from Seattle --count 1000</code>

Each layer builds on the one below.

▲ more control
more realism

▼
Use as much — or as little — as the test needs.

LAYER 1 • SCHEMA-DRIVEN

Random — but valid.

- ▶ **SchemaBasedFhirResourceFaker** — generates any FHIR resource type from schema metadata, not JSON templates
- ▶ **Binding-aware** — picks codes from the correct value sets, not random strings
- ▶ **Semantically valid** — a Patient gets a valid gender code; an Observation a valid LOINC code
- ▶ **All FHIR versions** — STU3, R4, R4B, R5, R6

BindingCodeMapper

value set → *terminology*

observation-codes → LOINC

condition-code → SNOMED CT

medication-codes → RxNorm

vaccine-code → CVX

procedure-code → SNOMED CT

allergyintolerance → SNOMED CT

Not just structurally valid — semantically valid.

THE HARD PART

Clinical correlation is a design constraint.

Random ≠ realistic. Vitals can't be independent random values.

- ▶ BMI must correlate with obesity-related conditions
- ▶ Blood glucose must track the diabetes diagnosis
- ▶ HbA1c > 6.5 if diabetic; systolic BP up with hypertension
- ▶ Lab values must be physiologically plausible together

The FHIR spec tells you the **schema**.

Clinical medicine tells you the **constraints**.

BMI 18 + type-2 diabetes + normal glucose is valid FHIR — and useless for testing. Any test relying on correlation would pass for the wrong reason.

VitalSignCorrelationEngine models the physiology explicitly.

LAYER 1 IN PRACTICE

Three lines.

```
var schema = FhirVersion.R5.GetSchemaProvider();  
var faker  = new SchemaBasedFhirResourceFaker(schema);  
  
var patient = faker.Generate("Patient");  
  
// Valid R5 Patient – correct structure,  
// valid coded values
```

LAYER 2 • FLUENT BUILDERS

Express clinical intent in C#.

ScenarioBuilder is a fluent composition API. You don't construct JSON — you describe what happened to a patient.

- ▶ `.WithPatient(p => p.WithAge(55))`
- ▶ `.AddEncounter("Annual wellness visit")`
- ▶ `.AddSubScenario(CommonScenarios.LipidPanel())`
- ▶ `.AddProbabilisticBranch(0.15, ...)`

The fluent API reads like a clinical protocol.

BEHIND THE FLUENT CALLS

ProbabilisticBranchState`.AddProbabilisticBranch(...)`

15% chance of hyperlipidemia — matches real-world prevalence

VitalSignCorrelationEngine`// applied automatically`

BMI correlates with conditions; labs stay physiologically plausible

CommonScenarios`CommonScenarios.LipidPanel()`

Reusable fragments: RecordVitalSigns, LipidPanel, BloodGlucose

LAYER 2 IN PRACTICE

Fluent syntax for complex scenario building.

```
var ctx = new ScenarioBuilder(schema)
    .WithPatient(p => p.WithAge(55).WithGender(g => g.Male))
    .AddEncounter("Annual wellness visit")
    .AddSubScenario(CommonScenarios.RecordVitalSigns())
    .AddSubScenario(CommonScenarios.LipidPanel())
    .AddProbabilisticBranch(
        probability: 0.15,
        trueState: new ConditionOnsetState { Code = Conditions.Hyperlipidemia })
    .Build();
```

55-year-old male · wellness visit · vitals · 15% hyperlipidemia — correlated, not random.

LAYER 3 · PATIENT LIFECYCLES

A full life, simulated.

- ▶ **PatientLifecycleGenerator** — age-based event scheduling, birth to today
- ▶ **DiseaseRiskCalculator** — evidence-based, age-adjusted onset probabilities
- ▶ **Wellness schedules** — realistic visit frequency, pediatric vs adult

PRE-BUILT ARCHETYPES

- ▶ Healthy child (0–18)
- ▶ Typical adult (0–45)
- ▶ Pediatric asthma · Elderly multi-morbidity

ARCHETYPE · METABOLIC SYNDROME

0 → 50 years, full trajectory**40-45**

encounters

3-6

medications

150+

vitals & labs

100+

total resources

Childhood visits → obesity → diabetes → escalation.

LAYER 3 IN PRACTICE

One patient. A full lifecycle.

 C#

```
var lifecycle = new PatientLifecycleGenerator(schemaProvider)
    .WithBirthYear(1980)
    .WithGender("female")
    .WithGivenName("Jennifer")
    .WithFamilyName("Martinez")
    .AddWellnessSchedule(pediatric: true, adult: true)
    .AddImmunizationSchedule()

// Probabilistic Type 2 Diabetes (15% overall probability ages 40-65)
// Uses age-dependent risk calculation: age 45 baseline ~10%
.AddProbabilisticCondition(
    "Type 2 Diabetes",
    onsetAges: 40..65,
    probability: 0.15,
    scenarioFactory: sp => new ScenarioBuilder(sp)
        .AddConditionOnset(FhirCode.Conditions.DiabetesType2, severity: 2)
        .AddMedicationOrder(FhirCode.Medications.Metformin500mg, isChronic: true, frequency: "BID", reasonCode: FhirCode.Conditions.DiabetesType2))
```

LAYER 4 · POPULATIONS

Model populations at scale.

- ▶ **PopulationGenerator** — large-scale cohort creation
- ▶ **11 cities** — 2020 demographics
- ▶ **EthnicNameGenerator** — culturally appropriate names via Bogus locales
- ▶ **Full history per patient** — birth to current age

Boston ≠ Seattle ≠ Amsterdam

different age, race & disease prevalence — the risk model is demographically aware

CityDemographics

City	Pop.	0–17	65+
New York	8.3M	21%	10%
Chicago	2.7M	21%	9%
Boston	676K	17%	12%
Seattle	737K	16%	12%

Boston race split: 53% White · 25% Black · 19% Hispanic · 9% Asian

DEMO 1

Let's generate some patients.

resource → scenario → population

```
$ ignixa-fakes r4 population --from Seattle --count 50 --ndjson --out ./output
```

CLI REFERENCE

One tool, every shape of data.

Command	Output
<code>ignixa-fakes r4 resource Patient</code>	Single Patient JSON
<code>ignixa-fakes r4 resource Observation BloodGlucose</code>	Observation with predefined state
<code>ignixa-fakes r4 scenario DiabeticPatient</code>	Bundle with full clinical scenario
<code>ignixa-fakes r4 population --from Seattle --count 100</code>	Transaction bundle — temp urn:uuid refs
<code>ignixa-fakes r4 population --count 100 --resolved-references</code>	Resolved batch bundle — real IDs
<code>ignixa-fakes r4 population --count 100 --ndjson</code>	NDJSON — ready to test \$import
<code>ignixa-fakes help scenarios</code>	List all available scenarios
<code>ignixa-fakes help cities</code>	List cities + demographics

Known limits & roadmap.

1	Infrastructure & metadata Conformance & definition resources	CapabilityStatement · StructureDefinition · SearchParameter
2	Workflow & decision support Plan / measure / action structures	PlanDefinition · Measure · ActivityDefinition
3	Financial & billing Nested Money-type cardinality	ClaimResponse · ExplanationOfBenefit · PaymentNotice
4	Testing & scenarios Deep setup / teardown nesting	TestScript · TestReport · Questionnaire

STILL COVERED

Patient, Observation,
Encounter —
unaffected.

**Validated STU3–R6.
Money &
ElementDefinition
next. #119**

GET STARTED

Try it.

```
# library
$ dotnet add package Ignixa.FhirFakes
# CLI
$ dotnet tool install -g Ignixa.FhirFakes.Cli
# generate your first patient
$ ignixa-fakes r4 resource Patient --from Seattle --out ./output
```

GitHub

github.com/brendankowitz/ignixa-fhir

Docs

brendankowitz.github.io/ignixa-fhir

License

MIT

Contact

You can find / reach me here:



Speaker's Gallery

Via the DevDays app



Email

bkowitz@microsoft.com



Bluesky

[@brendankowitz.bsky.social](https://bsky.app/profile/brendankowitz.bsky.social)



GitHub

github.com/brendankowitz/ignixa-fhir

Q&A

Feedback

- Enjoyed this talk? I'd love your feedback — it genuinely helps.
Scan the QR code, or visit:
<https://www.devdays.com/feedback/>

